CHAPTER – 1

INTRODUCTION

1.1 ABOUT THE PROJECT

LexiGuard AI – Enterprise Smart Contract Risk Intelligence System is an advanced AI-powered platform developed to simplify and automate the process of contract review and risk identification. In modern organizations, contracts play a crucial role in defining business relationships, partnerships, employment agreements, vendor management, and compliance requirements. However, most legal contracts are written using complex legal terminology and lengthy clause structures, making them difficult for individuals and non-legal professionals to understand. This complexity increases the chances of overlooking important clauses that may expose organizations to legal, financial, or operational risks.

The LexiGuard AI system is designed to address these challenges by integrating artificial intelligence, automated analysis, and interactive visualization into a single web-based platform. The application allows users to upload contract documents or enter contract text directly into the system. Once the contract is submitted, the system processes the document and performs clause-level analysis to identify important contractual components such as payment terms, liability clauses, termination conditions, confidentiality agreements, indemnity provisions, and governing laws.

Each clause is categorized into different risk levels such as Low Risk, Medium Risk, and High Risk. This classification helps users quickly identify critical or potentially harmful sections within the contract. By highlighting risky clauses, the system enables organizations and individuals to review and understand potential legal exposure before signing agreements. This automated risk assessment reduces the dependency on manual contract review and improves the overall efficiency of the contract evaluation process.

LexiGuard AI also integrates an AI-powered Legal Assistant using the OpenAI API, which acts as an intelligent chatbot capable of answering user questions related to contract clauses and legal concepts. Users can interact with the assistant to ask about specific clauses, request explanations of legal terms, or understand the implications of certain contract conditions. The AI assistant provides structured and simplified explanations, making complex legal language easier to understand for users who may not have a legal background.

The system further enhances contract understanding through an interactive dashboard that visually represents contract risk distribution and analytics. The dashboard displays metrics such as the number of clauses analyzed, risk distribution across categories, and graphical representations including pie charts and bar graphs. These visualizations help users quickly interpret the overall risk profile of the contract and make informed decisions.

Another important feature of the system is the automated report generation capability. After completing the contract analysis, LexiGuard AI generates a structured and professional PDF report that includes the clause-level risk assessment, analysis summary, and key insights. These reports can be downloaded, shared, and stored for documentation, compliance verification, or future reference by organizations and legal teams.

The project is implemented using Python and Streamlit, enabling the development of an interactive web application with minimal complexity. Additional technologies such as Pandas for data processing, Plotly for data visualization, and ReportLab for PDF generation are integrated to enhance the functionality of the system. The combination of these technologies allows the platform to deliver efficient performance, clear visualization, and automated reporting capabilities.

Overall, LexiGuard AI aims to transform traditional contract management by introducing intelligent automation into the contract review process. By combining artificial intelligence, data analytics, and user-friendly interfaces, the system significantly improves the speed, accuracy, and accessibility of contract analysis. It helps legal professionals, businesses, and individuals better understand contractual risks, reduce manual effort, and make more confident and informed legal decisions.

1.2 OBJECTIVE OF THE PROJECT

- **To Develop an Intelligent Contract Analysis System**

The main objective of the LexiGuard AI project is to develop an intelligent system that can automatically analyze contract documents and identify important clauses. The system helps users detect potential legal risks by classifying clauses into different risk levels such as low, medium, and high risk.

- **To Simplify Complex Legal Language**

Another objective of the project is to simplify complex legal terminology present in contracts. The

system uses an AI-powered legal assistant to provide clear explanations of contract clauses so that users without legal expertise can understand the meaning and implications of the terms.

- **To Provide AI-Based Legal Assistance**

The project aims to integrate an AI assistant that can answer user queries related to contract law, explain clauses, and provide guidance on legal concepts. This helps users gain better insight into contract terms and conditions.

- **To Visualize Contract Risk through Dashboards**

The system provides interactive dashboards that display contract risk analytics through charts and metrics. This allows users to quickly understand the distribution of risk levels across the contract.

- **To Generate Automated Contract Reports**

Another objective is to automatically generate structured PDF reports containing contract analysis results, clause-level risk classification, and summary insights. These reports can be downloaded and used for documentation and future reference.

- **To Improve Efficiency and Reduce Manual Effort**

The system aims to reduce the time and effort required for manual contract review by automating the analysis process and providing quick, accurate results.

- **To Support Better Decision Making**

By providing risk analysis, legal explanations, and structured reports, the system helps organizations and individuals make informed decisions before signing or approving contracts.

1.3 PROBLEM STATEMENT

- Business contracts often contain complex legal language and lengthy clauses that are difficult for non-legal professionals to understand. This makes it challenging for users to identify important terms and potential risks within the document.

- Traditional contract analysis is performed manually by legal professionals, which requires significant time and effort. Reviewing large contracts clause by clause can slow down business processes and delay decision-making.

- Manual contract review may lead to human errors such as overlooking critical clauses related to liability, termination, payment conditions, or compliance requirements. Missing such clauses can lead to legal disputes and financial losses.

- Many individuals, startups, and small organizations may not have access to experienced legal professionals for reviewing contracts. This creates a gap in understanding contractual obligations and risks.

- Most traditional contract management systems do not provide automated clause-level risk detection or intelligent insights, making it difficult to quickly assess the risk level of different contract provisions.Users often struggle to interpret legal terminology and complex clauses.

- Many contract review processes lack proper dashboards, analytics, and automated reports that summarize contract risks and insights in an easy-to-understand format.

1.4 MODULE DESCRIPTION

- User Authentication and Access Control
- Contract Upload and Data Processing
- Contract Risk Analysis and Clause Classification
- AI Legal Assistant
- Analytics Dashboard and Report Generation

1.4.1 User Authentication and Access Control

This module is responsible for secure user login and access management within the LexiGuard AI system. It ensures that only authorized users can access the contract analysis platform. Users must enter valid username and password credentials to log into the system. The module manages session state authentication, ensuring that users remain logged in during their interaction with the application. It also restricts unauthorized access to system features such as contract upload, analysis, and report generation. By implementing authentication control, the system ensures data privacy, security, and controlled access to sensitive contract information.Additionally, the module provides a structured login interface, allowing users to securely enter their credentials and access the dashboard after successful authentication.

1.4.2 Contract Upload and Data Processing

The Contract Upload and Data Processing module allows users to upload or input contract documents for analysis. Users can upload contract files or paste contract text directly into the system interface.Once the contract is uploaded, the system performs text preprocessing and structuring to prepare the document for analysis.

This includes removing unnecessary spaces, formatting the text, identifying clauses, and converting the contract into a structured format suitable for risk evaluation.The processed contract data is then passed to the analysis engine, where individual clauses are identified and prepared for further risk classification. This module ensures that contract data is clean, organized, and ready for automated processing.

1.4.3 Contract Risk Analysis and Clause Classification

This module is the core analytical component of the LexiGuard AI system. It performs clause-level analysis to detect important contractual provisions and identify potential risks.The system evaluates different types of clauses such as payment terms, termination clauses, liability clauses, confidentiality agreements, indemnity clauses, and governing law provisions. Each clause is analyzed and categorized into Low Risk, Medium Risk, or High Risk levels.

By automatically identifying risk-prone clauses, the system helps users understand which sections of the contract require careful attention. This automated risk classification significantly reduces the time and effort required for manual contract review and improves the accuracy of risk detection.

1.4.4 AI Legal Assistant

The AI Legal Assistant module provides intelligent legal support using OpenAI-powered AI technology. This module acts as a chatbot that helps users understand contract clauses and legal terminology.Users can ask questions related to contract terms, risk explanations, and legal concepts. The AI assistant analyzes the user query and provides structured and simplified explanations of the relevant legal information.

This module helps users who may not have a legal background to better understand contractual obligations and risks. It enhances the system by offering interactive guidance and real-time legal insights, making contract analysis more accessible and informative.

1.4.5 Analytics Dashboard and Report Generation

The Analytics Dashboard module provides users with a visual overview of contract risk analysis results. The dashboard displays risk metrics, charts, and summary statistics related to the analyzed contract.Interactive visualizations such as pie charts and bar graphs show the distribution of clauses across different risk levels.

These visual insights help users quickly understand the overall risk profile of the contract.In addition to visualization, this module also generates automated PDF reports containing detailed contract analysis results. The reports include clause-level risk classification, analysis summaries, and key insights. Users can download these reports for documentation, legal review, or future reference.

CHAPTER – 2

SYSTEM SPECIFICATION

2.1 HARDWARE SPECIFICATION

- Processor: AMD Ryzen 5000 series
- RAM: 8 GB
- Hard Disk Drive: 512 GB
- Printer: HP Ink Jet
- Keyboard: HP
- Mouse: Logi Tech (Optical)

2.2 SOFTWARE SPECIFICATION

- Front End / GUI Tool: Streamlit (with embedded HTML and CSS)
- Operating System: Windows 11
- Coding Language: Python
- Back End: Python (Streamlit Framework)
- Libraries Used: Pandas, Plotly, ReportLab
- AI Integration: OpenAI API

2.3 SOFTWARE DESCRIPTION

LexiGuard AI is developed entirely using Python and modern supporting libraries to create an intelligent, interactive, and enterprise-ready contract risk analysis system. The application integrates backend processing, AI capabilities, data visualization, and report generation within a single unified environment.

Python

Python is a high-level, interpreted, object-oriented programming language created by Guido van Rossum in 1989. It is widely used in artificial intelligence, automation, web development, and data analytics due to its simplicity, readability, and powerful ecosystem. Python supports dynamic

typing, automatic memory management, exception handling, and modular programming, which significantly improves development speed and maintainability.

In LexiGuard AI, Python serves as the core development language and controls the entire system workflow. It handles backend application logic, clause-level risk processing, AI API communication, session state management, file operations, and PDF generation. The modular nature of Python allows different functional components such as authentication, dashboard analytics, AI assistant processing, and reporting to be logically separated while remaining interconnected. Its scalability and flexibility make it highly suitable for enterprise-level intelligent applications.

Streamlit Framework

The application interface is developed using Streamlit, an open-source Python framework designed for building interactive web applications with minimal frontend coding. Streamlit enables developers to create fully functional web applications using only Python scripts, eliminating the need for separate HTML, CSS, or JavaScript frameworks.

In LexiGuard AI, Streamlit is responsible for rendering the complete user interface and managing user interactions. It provides page configuration, sidebar navigation, file upload handling, interactive dashboards, metric displays, chat interfaces, session state management, and report download functionality. Streamlit's reactive execution model ensures that the interface dynamically updates based on user inputs, thereby improving responsiveness and usability. Its lightweight architecture supports rapid development, easy testing, and smooth deployment in both local and cloud environments.

OpenAI API Integration

LexiGuard AI incorporates artificial intelligence capabilities using the API provided by OpenAI. The system integrates a GPT-based language model to power the AI Legal Assistant module, enabling the generation of structured legal explanations and contract-related insights in real time.

The AI module is implemented using secure API key handling mechanisms such as environment variables or Streamlit secrets. Structured prompt engineering guides the AI model toward contract law–specific responses and prevents unrelated outputs. Conversation history is maintained through session state memory to ensure contextual continuity and meaningful interactions. The AI assistant

provides clause explanations, legal references, risk clarifications, and structured answers in a professional format, significantly enhancing the intelligence and analytical depth of the system.

Pandas Library

Data manipulation and structured analysis within LexiGuard AI are performed using the Pandas library. Pandas provides powerful DataFrame and Series data structures that allow efficient handling of structured and tabular information.

In this system, Pandas is used to create clause-level datasets, categorize risk levels, manage structured risk tables, compute summary statistics, and present organized outputs. It transforms contract clause information into analytical data that can be easily visualized and interpreted. The use of Pandas ensures efficient data processing, faster computation, and smooth integration with visualization and reporting libraries.

Plotly

For data visualization, LexiGuard AI uses Plotly to generate interactive and visually appealing charts. Plotly enables the creation of dynamic graphs that enhance analytical interpretation and improve user engagement.

Within the dashboard and contract analysis modules, Plotly is used to generate risk distribution pie charts, bar charts, and other analytical visuals. These interactive visualizations allow users to quickly understand the distribution of risk levels and overall contract exposure. The integration of Plotly improves the system's decision-support capabilities and makes complex data easier to interpret.

ReportLab

PDF report generation is implemented using ReportLab, a Python library for programmatically creating PDF documents. In LexiGuard AI, ReportLab dynamically generates structured contract risk reports containing analysis summaries and risk findings.

This functionality allows users to download professional PDF reports directly from the system. The reporting module ensures that analysis results can be stored, documented, and shared in a standardized format suitable for enterprise environments and official documentation.

HTML and CSS (Embedded in Streamlit)

Although the application is primarily built using Streamlit, custom HTML and CSS are embedded within the Python code to enhance visual design and user experience. These technologies are used to implement a modern glassmorphism-style login interface, custom button styling, background image integration, layout adjustments, and hover effects.

The integration of HTML and CSS improves the aesthetic appeal and professionalism of the system without requiring a separate frontend framework. This approach maintains architectural simplicity while enhancing the overall interface design.

Software Architecture

LexiGuard AI follows a modular and layered architecture to ensure maintainability and scalability. The authentication module manages login validation and session-based access control. The dashboard module presents system metrics and interactive analytics. The contract analysis module performs clause-level risk categorization and displays explanatory insights. The AI assistant module provides GPT-powered structured legal responses, and the reporting module generates downloadable PDF summaries of contract evaluations.

This structured architecture enables clean separation of concerns, simplifies debugging, and allows future enhancements such as database integration, multi-user authentication, and advanced analytics.

Development Environment

The system is developed using Python 3.x and implemented with the Streamlit framework. Supporting libraries include Pandas for data handling, Plotly for visualization, and ReportLab for PDF generation, while AI capabilities are powered by OpenAI's GPT model. Development and testing were carried out using Visual Studio Code on a Windows operating system, ensuring compatibility, ease of debugging, and efficient project management

CHAPTER – 3

SYSTEM STUDY

3.1 EXISTING SYSTEM

In the existing system, contract analysis is primarily performed through manual review by legal professionals. Organizations depend on lawyers or legal consultants to read, interpret, and evaluate contractual documents. This process requires significant legal expertise, consumes considerable time, and increases operational costs. Manual contract screening is also prone to human errors, oversight of critical clauses, and inconsistency in risk interpretation.Some organizations use basic digital tools such as document management systems or keyword-based search tools to identify specific terms within contracts. However, these systems lack contextual understanding and cannot accurately evaluate the legal implications of clauses. They rely heavily on simple text matching techniques, which fail to assess risk severity, clause ambiguity, or imbalance between parties.Existing automated tools in the market may use rule-based approaches or limited machine learning techniques to extract certain contract elements. However, most of these systems provide only clause detection without offering structured risk classification, simplified explanations, or legal references. They often lack conversational assistance and do not provide comprehensive enterprise-level analytics or automated risk reporting.Therefore, the existing system is time-consuming, costly, less scalable, and does not provide intelligent, real-time legal risk insights. These limitations highlight the need for an AI-driven contract risk intelligence system that can automate clause extraction, perform structured risk assessment, and support informed decision-making.

3.2 LIMITATION OF EXISTING SYSTEM

The existing contract review system heavily depends on manual legal analysis, which requires professional expertise and extensive time for document examination. This process is slow, expensive, and not scalable for organizations handling multiple contracts simultaneously. Manual review also increases the risk of human error, oversight of critical clauses, and inconsistent interpretation of legal terms.

Most traditional digital tools used in contract management rely on basic keyword search or rule-based methods. These systems lack contextual understanding and cannot accurately evaluate clause meaning, legal intent, or risk severity. They are unable to classify clauses into structured risk

categories such as low, medium, or high risk. As a result, they provide limited analytical value and fail to deliver actionable insights.Additionally, existing systems do not offer simplified legal explanations for non-legal users. Contracts often contain complex terminology that business professionals may struggle to understand. Without AI-driven interpretation, users must rely entirely on legal experts for clarification. Furthermore, many available tools lack conversational AI support, real-time analytics dashboards, and automated report generation features, making them insufficient for enterprise-level contract intelligence and decision-making.

3.3 PROPOSED SYSTEM

To overcome the drawbacks of the methods we have reviewed above, we propose a new model for sentiment analysis. In this model we combine many techniques to reach our final goal of emotion extraction. The steps for the process are documented below.

1. Retrieval of Data: data is mined using the existing for data extraction. Data's would be selected based on a few chosen keywords pertaining to the domain of our concern, i.e. product reviews. We have elected to use the API due to ease of data extraction.

2. Preprocessing: In this stage, the data is put through a preprocessing stage in which we remove identifying information such as data handles, timestamps of the message and embedded links and videos. Such information is largely irrelevant and may cause false results to be given by our system.

3. Word Correction: As text are written for human perusal, they often contain slang, misspellings and other irrelevant data. Thus we correct the misspellings in the sentences and look to replace the slang in the sentences with words from standard English that may roughly relate to the slang in question. As slang itself can be used to display a wide variety of sentiment, often with greater emotional impact, this process is necessary so that slang words may be considered as part of the emotion expressed.

Polarity detection: In this step we begin the second phase of our proposed system, in which we try to identify the polarity of the sentence in question. If emoticons exist in the statements, they will be used as well to compute the overall polarity of the statement. We aim to find sentences where the polarity detection is not very clear or where the expressed sentiment may be low. We also try to isolate the opinion words in the sentence in relation to a given concept in the sentence.

3.4 ADAVANTAGES OF PROPOSED SYSTEM

That's where sentiment analysis can help to:

- Understand what your customers like and dislike about your product.
- Compare your product reviews with those of your competitors.
- Get the latest product insights in real-time, 24/7.
- Save hundreds of hours of manual data processing.
- You can develop a more insightful, data-based marketing strategy
- Give extra boost to your customer service
- Find industry leaders and influencers and understand the customers.

CHAPTER – 4

SYSTEM DESIGN

4.1 INPUT DESIGN

The input design of LexiGuard AI focuses on converting user-provided contract data into a structured and system-compatible format while minimizing errors and improving overall usability. The application is menu-driven and developed using the Streamlit framework, which provides clearly labeled screens such as Login, Contract Upload, Dashboard, AI Assistant, and Report Generation. This structured interface guides users step by step, ensuring that the data entered into the system follows a consistent workflow and reduces the chances of confusion during operation.

Users can upload contract files in supported formats such as PDF or DOCX, or alternatively paste contract text directly into the system for quick analysis. The interface provides sufficient space for data entry and clear instructions to help users understand the required inputs. Multiple validation mechanisms are implemented to prevent incomplete or invalid submissions, including mandatory login fields, restricted file format checks, and session-based controls to avoid duplicate processing. The system also performs text preprocessing to remove unnecessary characters, formatting inconsistencies, and noise from the uploaded content, ensuring accurate clause identification and risk analysis.

In addition, the input design prioritizes user convenience and data reliability by providing feedback messages, error alerts, and guided prompts during the input process. These features help users correct mistakes quickly and ensure that only valid and meaningful data enters the system. Overall, the input design of LexiGuard AI emphasizes accuracy, simplicity, security, and logical data flow, making the system easy to use while ensuring high-quality data for analysis.

4.2 OUTPUT DESIGN

The output design of LexiGuard AI is focused on delivering clear, structured, and professional results that support effective decision-making for users. The system generates multiple types of outputs, including clause-level risk categorization, dashboard analytics, AI-generated legal explanations, and downloadable PDF reports.

These outputs are designed to transform complex legal information into simplified and easy-to-understand insights, enabling users to quickly identify potential risks within contracts.

On-screen outputs include interactive dashboards, structured tables, and visual charts that display the distribution of low, medium, and high-risk clauses, helping users gain an overall understanding of contract exposure. Each clause is accompanied by a brief explanation to clarify the reason for its risk level. The AI assistant also provides structured legal responses to user queries, enhancing understanding of contract terms and legal concepts. Additionally, the system generates a professionally formatted PDF report containing the contract summary, analysis level, and risk insights, which can be downloaded for documentation or sharing purposes. The outputs are designed for both digital viewing and printed documentation, ensuring clarity, usability, and compatibility with professional reporting standards.

4.3 TABLE DESIGN

Table design defines the structure of the database used to store and manage the information in the LexiGuard AI – Enterprise Smart Contract Risk Intelligence System. Proper table design ensures efficient storage, retrieval, and management of data related to users, contracts, clause analysis, and generated reports. The tables are structured to maintain data integrity and support the various modules of the system such as authentication, contract analysis, and report generation.

1. User Table

This table stores information about registered users who access the system.

Field Name	Data Type	Description
user_id	INT (Primary Key)	Unique ID for each user
username	VARCHAR(50)	Username for login
password	VARCHAR(100)	Encrypted password
email	VARCHAR(100)	User email address
role	VARCHAR(20)	User role (Admin/User)
created_at	DATETIME	Account creation date

2. Contract Table

This table stores information about uploaded contracts.

Field Name	Data Type	Description
contract_id	INT (Primary Key)	Unique contract ID
user_id	INT (Foreign Key)	ID of the user who uploaded the contract
contract_name	VARCHAR(200)	Name of the contract
contract_text	TEXT	Full contract content
upload_date	DATETIME	Date of upload

3. Clause Analysis Table

This table stores clause-level analysis results generated by the system.

Field Name	Data Type	Description
clause_id	INT (Primary Key)	Unique clause identifier
contract_id	INT (Foreign Key)	Associated contract ID
clause_text	TEXT	Extracted clause text
clause_type	VARCHAR(100)	Type of clause
risk_level	VARCHAR(20)	Risk classification (Low, Medium, High)
explanation	TEXT	Risk explanation provided by the system

4. AI Chat History Table

This table stores user interactions with the AI Legal Assistant.

Field Name	Data Type	Description
chat_id	INT (Primary Key)	Unique chat message ID
user_id	INT (Foreign Key)	User who asked the query
user_query	TEXT	Question asked by the user
ai_response	TEXT	Response generated by AI
timestamp	DATETIME	Date and time of interaction

5. Report Table

This table stores generated contract analysis reports.

Field Name	Data Type	Description
report_id	INT (Primary Key)	Unique report ID
contract_id	INT (Foreign Key)	Related contract ID
report_name	VARCHAR(200)	Generated report file name
report_path	VARCHAR(255)	File storage path
generated_date	DATETIME	Report generation date

CHAPTER – 5

SYSTEM TESTING

5.1 TESTING

System testing ensures that the LexiGuard AI – Enterprise Smart Contract Risk Intelligence System works correctly and satisfies all functional and non-functional requirements. The main objective of testing is to identify errors, improve system performance, and ensure reliability before the system is deployed. Since the application analyzes legal contracts and provides risk assessments, it is important that the system produces accurate and consistent results.The testing process focuses on validating key functionalities such as user authentication, contract upload and processing, clause risk classification, AI assistant responses, dashboard visualization, and PDF report generation.

Both manual testing and automated testing techniques are used to verify the correctness of each module. The system is tested under different conditions to ensure stability, security, and usability.Different testing approaches such as Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT) are conducted to confirm that the application functions efficiently and provides reliable contract analysis results.

5.1.1 Unit Testing

Unit testing is performed to verify individual components of the system. Each module such as login authentication, contract upload, clause extraction, risk classification, AI assistant functionality, and report generation is tested separately to ensure it works correctly. For example, the login module is tested to verify valid user credentials, while the contract upload module checks whether the system correctly accepts and processes contract text. Unit testing helps identify errors at an early stage and improves the reliability of the system.

- Login authentication and session state handling
- Sidebar navigation and page routing
- Dashboard metrics and chart generation
- Contract upload and clause risk table display
- AI Assistant API request and response handling
- PDF report generation and download functionality

5.1.2 Integration Testing

Integration testing ensures that all modules of the system work together without issues. It checks the interaction between modules such as contract processing, risk analysis, AI assistant communication, and dashboard visualization. This testing verifies that the data generated by the contract analysis module is correctly passed to the dashboard and report generation modules. It also ensures that the AI assistant receives proper context for answering user queries.

- Login module → Main application access
- Sidebar navigation → Module switching
- File upload → Contract analysis workflow
- Chat input → OpenAI response display
- Report generation → File download process

5.1.3 System Testing

System testing evaluates the complete application as a whole. It checks the overall functionality, performance, security, and usability of the platform. Various real-world scenarios are tested, such as uploading large contracts, analyzing multiple clauses, generating reports, and interacting with the AI assistant. This stage ensures that the system performs efficiently and delivers accurate results under different user conditions.

5.1.4 User Acceptance Testing (UAT)

User Acceptance Testing is the final stage of testing where the system is evaluated by real users. Users interact with the platform by uploading contracts, viewing risk analysis results, and using the AI assistant. Feedback is collected regarding system usability, performance, and clarity of analysis results. Based on user feedback, necessary improvements are made before the final deployment of the system.

CHAPTER – 6

6.1 SYSTEM IMPLEMENTATION

6.1.1 System Deployment

The LexiGuard AI system is developed using Python and the Streamlit framework for building the web-based interface. The frontend interface is created using Streamlit components along with embedded HTML and CSS to provide an interactive and user-friendly design. The system integrates OpenAI API for AI-powered legal assistance and contract clause explanations.

The application follows a modular architecture, where different components such as authentication, contract analysis, AI assistant, dashboard visualization, and report generation operate together. The system can be deployed on a local server or cloud-based platform for easy access and scalability. Version control tools such as GitHub are used to manage project updates and maintain code consistency during development and deployment.

6.1.2 Database Setup

The system uses lightweight data storage mechanisms to manage information such as user credentials, contract text, clause analysis results, and generated reports. Data structures are organized to maintain proper relationships between contract inputs and their corresponding risk analysis outputs.

To maintain data integrity and efficient retrieval, structured storage techniques are used. Important information such as contract clauses, risk categories, and AI responses is processed and stored temporarily during system execution. Proper validation methods ensure that uploaded contract files and user inputs are correctly handled by the system.

6.1.3 AI Module Integration

The AI module is one of the core components of the LexiGuard AI system. It integrates the OpenAI API to provide intelligent legal assistance and contract clause explanations. The AI assistant analyzes user queries related to contracts and generates structured responses that help users understand complex legal terminology.

The AI module also assists in explaining clause meanings, clarifying legal concepts, and answering contract-related questions. Through prompt-based interaction, the system provides clear and

simplified explanations of contract provisions. This integration improves accessibility to legal insights and helps users make informed decisions during contract evaluation.

6.1.4 Security Measures

Security is an important aspect of the LexiGuard AI system since it handles sensitive contract information. The system includes secure login authentication to ensure that only authorized users can access the platform.Session management is implemented to maintain user login sessions and prevent unauthorized access. Input validation techniques are used to ensure that uploaded files and text inputs are safe and properly formatted. Additionally, secure API communication is maintained when interacting with external services such as the OpenAI API. These security mechanisms help protect user data and maintain system reliability.

6.1.5 User Training

Users are provided with simple instructions and guidance to help them understand how to operate the LexiGuard AI system effectively. Training materials include step-by-step usage instructions, interface explanations, and example workflows for contract analysis.Users learn how to log into the system, upload contracts, analyze clause risks, interact with the AI assistant, and download reports. This training helps users quickly become familiar with the platform and utilize its features efficiently.

6.1.6 Post-Deployment Monitoring

After deployment, the system is monitored to ensure smooth operation and performance stability. Monitoring involves checking application functionality, analyzing system logs, and identifying potential errors or performance issues.

User feedback is also collected to improve system features and usability. Regular updates are applied to improve performance, update AI responses, and enhance overall functionality. Continuous monitoring ensures that the system remains reliable and effective for contract analysis tasks.

6.2 SYSTEM MAINTENANCE

6.2.1 Corrective Maintenance

Corrective maintenance involves identifying and fixing errors or bugs that occur after the system is deployed. If issues such as incorrect risk classification, system crashes, or login errors occur, they are detected through debugging and system monitoring tools.Developers analyze the problem, correct the code, and apply updates to ensure that the system continues to operate smoothly. User feedback also plays an important role in identifying issues that require correction.

6.2.2 Adaptive Maintenance

Adaptive maintenance ensures that the system continues to function properly when technological environments or user requirements change. Updates may include improvements to AI models, API integration updates, compatibility with new operating systems, or enhancements to visualization tools.This type of maintenance allows the system to remain compatible with evolving technologies and ensures that the application continues to meet user needs.

6.2.3 Perfective Maintenance

Perfective maintenance focuses on improving system performance and enhancing features based on user feedback. This includes optimizing the contract analysis process, improving dashboard visualizations, and enhancing the accuracy of risk classification.User interface improvements, faster processing of contract data, and enhanced AI responses may also be implemented to improve the overall user experience and system efficiency.

6.2.4 Preventive Maintenance

Preventive maintenance aims to reduce the possibility of future system failures. This includes regular system monitoring, performance checks, and security updates. Backup procedures are also implemented to protect important data and ensure system recovery in case of unexpected failures.By performing regular maintenance and updates, the LexiGuard AI system remains stable, secure, and efficient over time.

CHAPTER – 7

CONCLUSION

The LexiGuard AI – Enterprise Smart Contract Risk Intelligence System represents a significant advancement in the field of automated contract analysis by integrating artificial intelligence, data processing, and interactive visualization. The system is designed to simplify the complex process of contract review by automatically analyzing contractual clauses and identifying potential legal risks. By leveraging modern technologies such as Python, Streamlit, OpenAI API, Pandas, Plotly, and ReportLab, the platform provides an intelligent and efficient solution for contract risk management.A key strength of the system lies in its ability to perform clause-level risk classification, where contract clauses are categorized into Low Risk, Medium Risk, and High Risk levels. This helps users quickly identify critical sections within a contract that may require further attention. The automated analysis significantly reduces the time and effort required for manual contract review while improving the accuracy and consistency of risk detection.Another important feature of the system is the AI-powered Legal Assistant, which allows users to interact with the system and receive simplified explanations of complex legal terms and clauses.

This functionality helps individuals who may not have extensive legal knowledge to better understand contractual obligations and implications. By providing structured responses and guidance, the AI assistant enhances accessibility to legal insights and supports informed decision-making.The system also provides an interactive dashboard that visually represents contract risk distribution through charts and metrics. These visualizations help users quickly understand the overall risk profile of a contract. In addition, the platform supports automated PDF report generation, enabling users to download structured contract analysis reports for documentation, compliance, and future reference.Security and reliability are also important aspects of the system. The implementation of secure user authentication, session management, and controlled access mechanisms ensures that sensitive contract data is protected. The modular architecture of the system allows easy maintenance, scalability, and future enhancements.

CHAPTER – 8

FUTURE ENHANCEMENT

The current version of LexiGuard AI functions as a working prototype for smart contract risk analysis and AI-based legal assistance, but there is significant scope for future enhancement and expansion. One of the major improvements would be the integration of advanced Natural Language Processing (NLP) techniques to automatically extract and analyze clauses directly from real uploaded contracts instead of using simulated data. This would make the system more practical and accurate for real-world legal environments. Additionally, integrating a database would allow the application to securely store contracts, analysis history, and generated reports for long-term access and tracking.

In the future, the system can be expanded to include multi-user authentication and role-based access control, enabling organizations to manage different user roles such as legal analysts, administrators, and managers. Cloud deployment using platforms like AWS or Azure would improve scalability, accessibility, and performance. The AI assistant can also be enhanced with multilingual support, deeper legal knowledge, and advanced risk scoring using machine learning. With these upgrades, LexiGuard AI has the potential to evolve into a comprehensive enterprise-level legal technology platform for efficient contract management and risk assessment.

CHAPTER – 9

9.1BOOK REFERENCE

1. Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education.
2. Jurafsky, D., & Martin, J. H. (2021). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. New Jersey: Pearson.
3. Bishop, C. M. (2016). Pattern Recognition and Machine Learning. Berlin: Springer.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. Cambridge: MIT Press.
5. McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. O'Reilly Media.
6. Grus, J. (2019). Data Science from Scratch: First Principles with Python (2nd ed.). O'Reilly Media.
7. VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.
8. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.

9.2 WEB REFERENCE

1) OpenAI API Documentation – Provides guidelines for integrating AI models for natural language understanding and chatbot development.
   https://platform.openai.com/docs
2) Streamlit Documentation – Official documentation for building interactive Python web applications and dashboards.
   https://docs.streamlit.io
3) Pandas Documentation – Guide for data manipulation and analysis using the Pandas Python library.
   https://pandas.pydata.org/docs
4) Plotly Documentation – Provides tools for creating interactive charts and dashboards in Python.
   https://plotly.com/python/
5) ReportLab Documentation – Python library used for generating PDF documents programmatically.
   https://www.reportlab.com/docs/

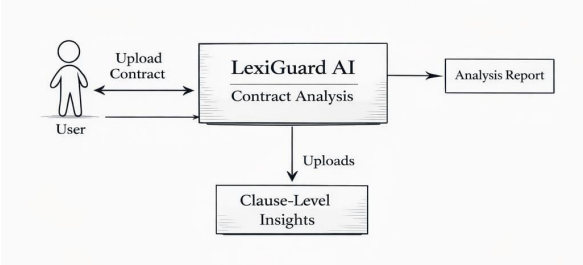
6) Python Official Documentation – Comprehensive documentation for the Python programming language.
https://docs.python.org/3/
7) Introduction to Natural Language Processing – Overview of NLP techniques used in AI applications.
https://towardsdatascience.com/natural-language-processing
8) AI Applications in Legal Technology – Discusses how artificial intelligence is transforming legal analysis and contract review.
https://www.ibm.com/topics/artificial-intelligence
9) OpenAI API Documentation – Provides guidelines for integrating AI models for natural language understanding and chatbot development.
https://platform.openai.com/docs
10) Streamlit Documentation – Official documentation for building interactive Python web applications and dashboards.
https://docs.streamlit.io
11) Pandas Documentation – Guide for data manipulation and analysis using the Pandas Python library.
https://pandas.pydata.org/docs
12) Plotly Documentation – Provides tools for creating interactive charts and dashboards in Python.
https://plotly.com/python/

CHAPTER – 10

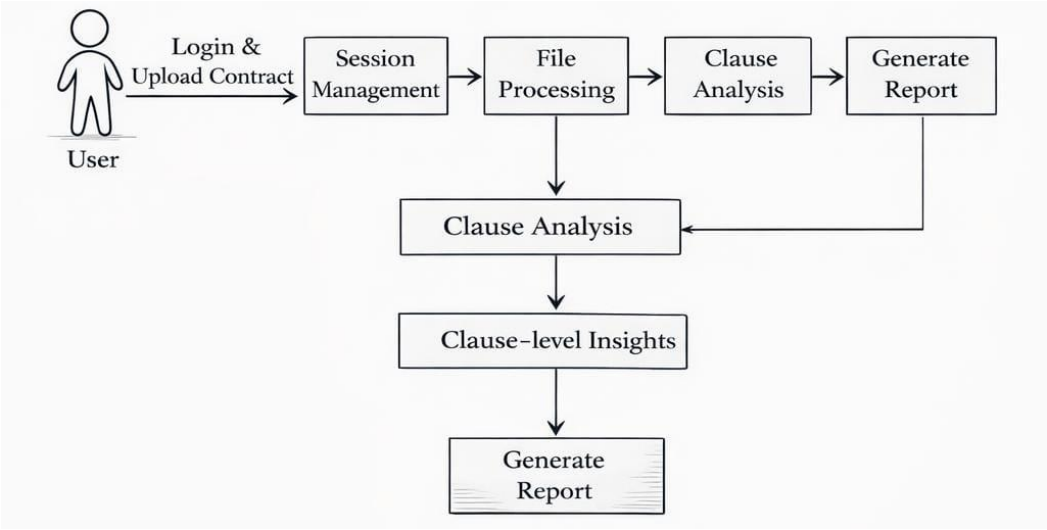
APPENDIX

A. DATA FLOW DIAGRAM

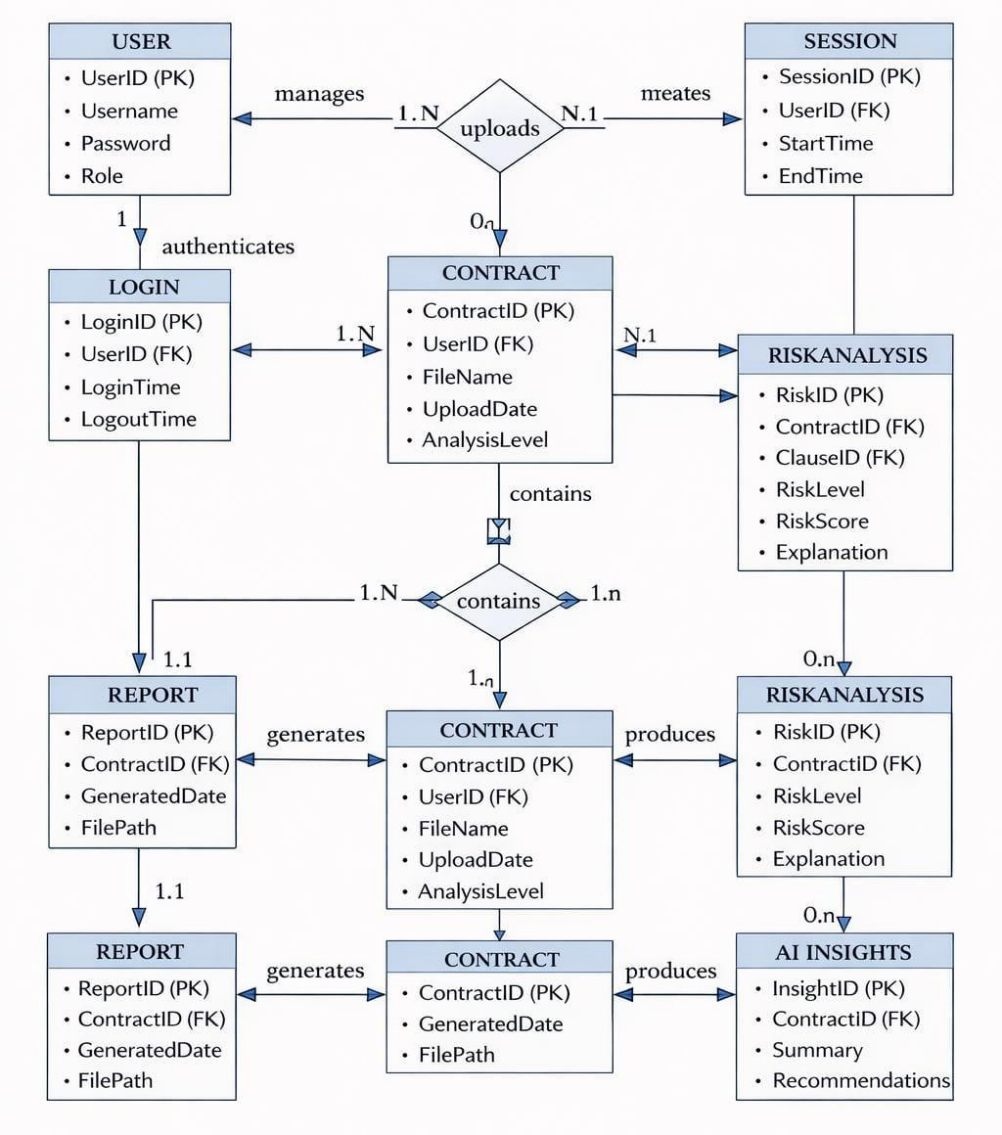
LEVEL 0 : Context Diagram



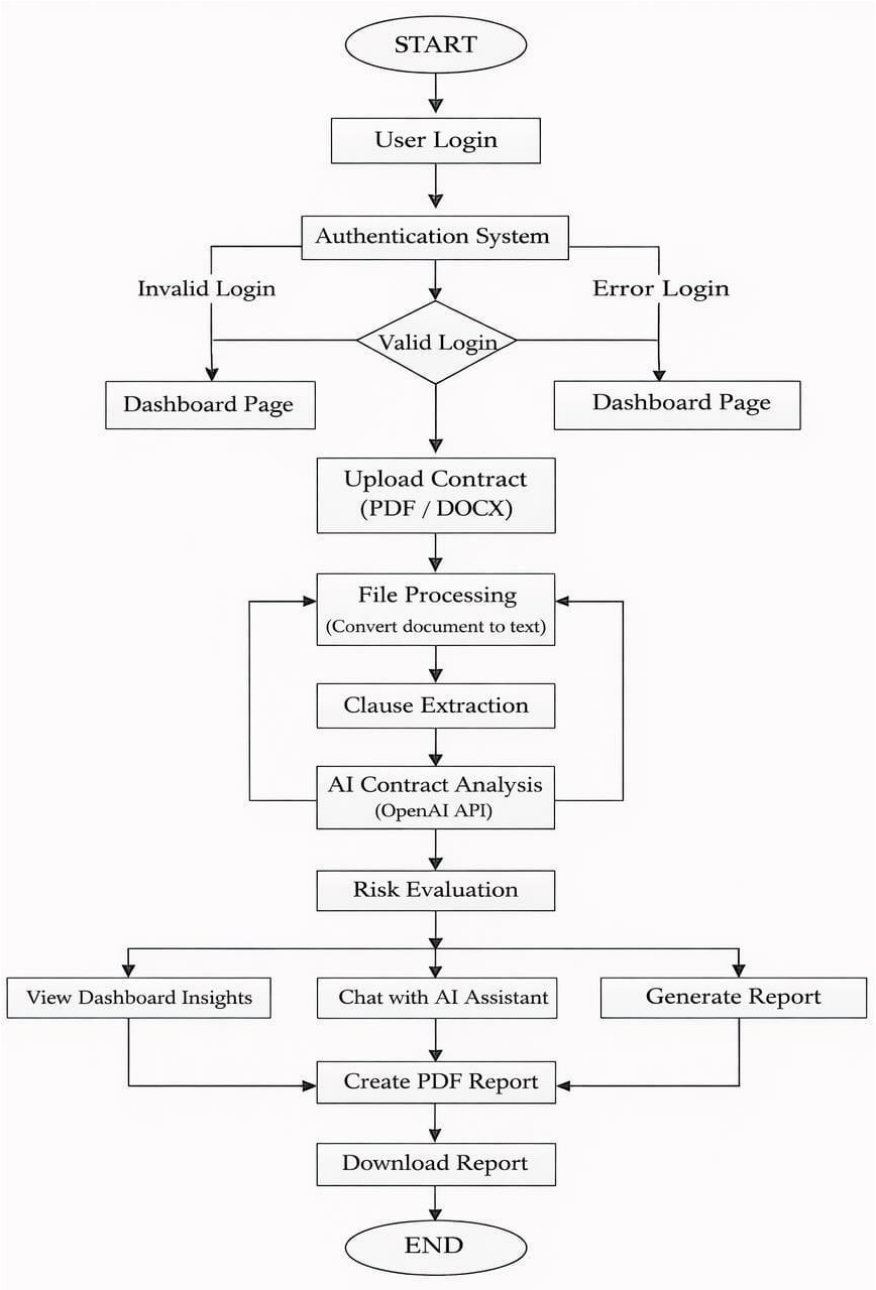
LEVEL 1 : High-Level



B.ENTITY RELATIONSHIP DIAGRAM



C. SYSTEM FLOW DIAGRAM



CODE

```
import streamlit as st
import pandas as pd
import plotly.express as px
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
import os
import datetime
from openai import OpenAI
import base64

# ---------------------------------------------------
# PAGE CONFIG
# ---------------------------------------------------
st.set_page_config(page_title="LexiGuard AI", layout="wide")

# ---------------------------------------------------
# LOAD CSS
# ---------------------------------------------------
def load_css():
    with open("assets/style.css") as f:
        st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)

load_css()

# ---------------------------------------------------
# SESSION LOGIN STATE
# ---------------------------------------------------
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False

# ---------------------------------------------------
# OPENAI CLIENT
```

```
# ---------------------------------------------------
api_key = st.secrets.get("OPENAI_API_KEY", None) or os.getenv("OPENAI_API_KEY")
client = OpenAI(api_key=api_key) if api_key else None

# ===================================================
# LOGIN PAGE — GLASS UI
# ===================================================
if not st.session_state.logged_in:

    def get_base64(file):
        try:
            with open(file, "rb") as f:
                return base64.b64encode(f.read()).decode()
        except:
            return ""

    bg_img = get_base64("logo.png")

    st.markdown(f"""
    <style>
    .stApp {{
        background-image: url("{bg_img}");
        background-size: cover;
        background-position: right;
        background-attachment: fixed;
    }}
    .login-card {{
        backdrop-filter: blur(30px);
        border-radius: 20px;
        padding: 40px;
        width: 320px;
        text-align: center;
        box-shadow: 0 8px 32px rgba(0,0,0,0.3);
    }}
```

```
    .login-title {{
        font-size: 40px;
        font-weight: 700;
        color: white;
        margin-bottom: 25px;
    }}
    label {{ color: white !important; }}
    </style>
    """, unsafe_allow_html=True)

    col1, col2, col3 = st.columns([1,1.2,1])

    with col2:
        st.markdown('<div class="login-card">', unsafe_allow_html=True)
        st.markdown('<div class="login-title">LexiGuard Sign In</div>', unsafe_allow_html=True)

        username = st.text_input("Username")
        password = st.text_input("Password", type="password")

        if st.button("Login"):
            if username == "admin" and password == "1234":
                st.session_state.logged_in = True
                st.rerun()
            else:
                st.error("Invalid Credentials")

        st.markdown('</div>', unsafe_allow_html=True)

# ==================================================
# MAIN APPLICATION
# ==================================================
else:

    st.title("LexiGuard AI")
```

```
st.subheader("Enterprise Smart Contract Risk Intelligence")

# ---------------- SIDEBAR ----------------
with st.sidebar:
    st.markdown("## LexiGuard AI")

    page = st.radio(
        "Navigation",
        ["Dashboard","Contract Analysis","AI Assistant","Reports"]
    )

    uploaded_file = st.file_uploader(
        "Upload Contract",
        type=["pdf","docx"]
    )

    analysis_level = st.selectbox(
        "Analysis Depth",
        ["Basic","Advanced","Legal Audit"]
    )

    st.write("User: Analyst")
    st.write("Session:", datetime.datetime.now().strftime("%H:%M:%S"))

    if st.button("Logout"):
        st.session_state.logged_in = False
        st.rerun()

# ---------------- DASHBOARD ----------------
if page == "Dashboard":

    st.subheader("System Overview")

    c1,c2,c3,c4 = st.columns(4)
```

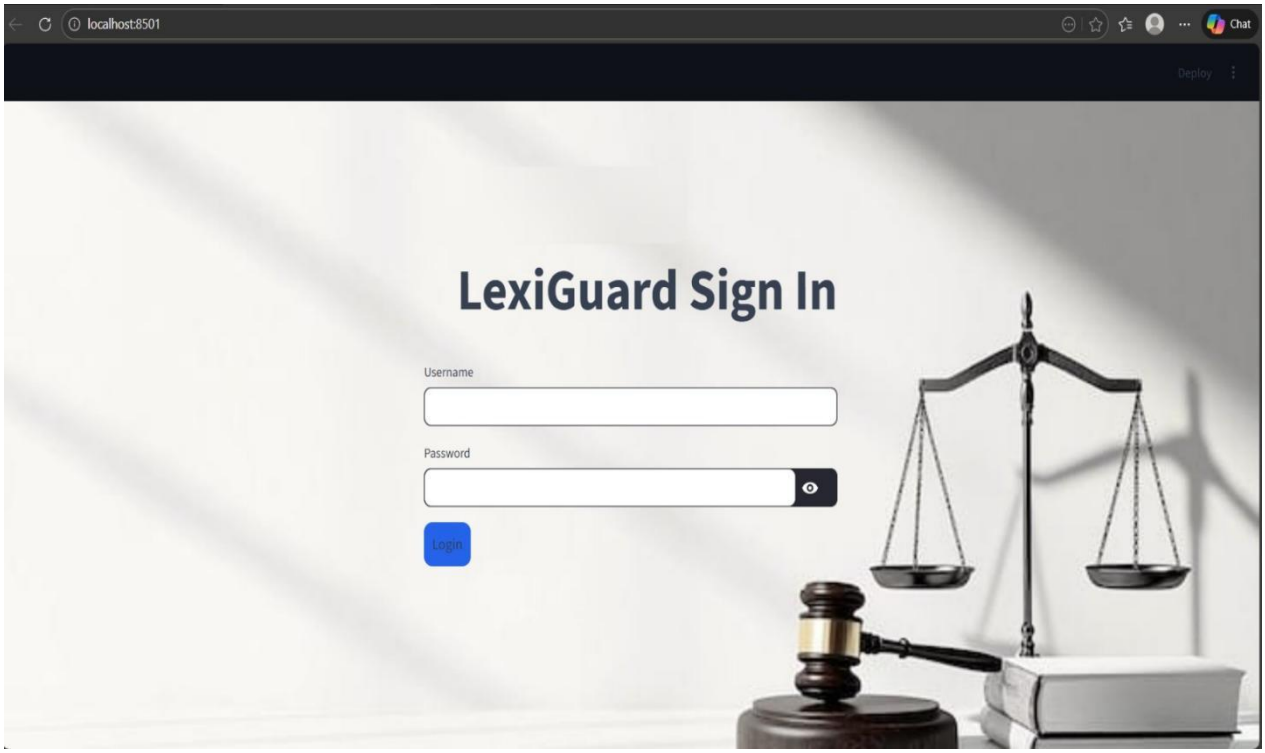
```
    c1.metric("Contracts","24","+3")
    c2.metric("Clauses","187")
    c3.metric("High Risk","12","-2")
    c4.metric("Confidence","92%")

    risk_data = pd.DataFrame({
        "Risk":["Low","Medium","High"],
        "Count":[120,45,22]
    })

    fig = px.bar(risk_data,x="Risk",y="Count",color="Risk")
    st.plotly_chart(fig,use_container_width=True)

# ---------------- CONTRACT ANALYSIS ----------------
elif page == "Contract Analysis":

    st.subheader("Clause-Level Risk Analysis")

    if uploaded_file:

        df = pd.DataFrame({
            "Clause":[
                "Payment Terms","Termination Clause","Liability Clause",
                "Confidentiality","Indemnity","Force Majeure",
                "Dispute Resolution","Data Protection",
                "Intellectual Property","Governing Law"
            ],
            "Risk Level":[
                "Low","Medium","High","Low","High",
                "Medium","Medium","Low","Medium","Low"
            ]
        })
```

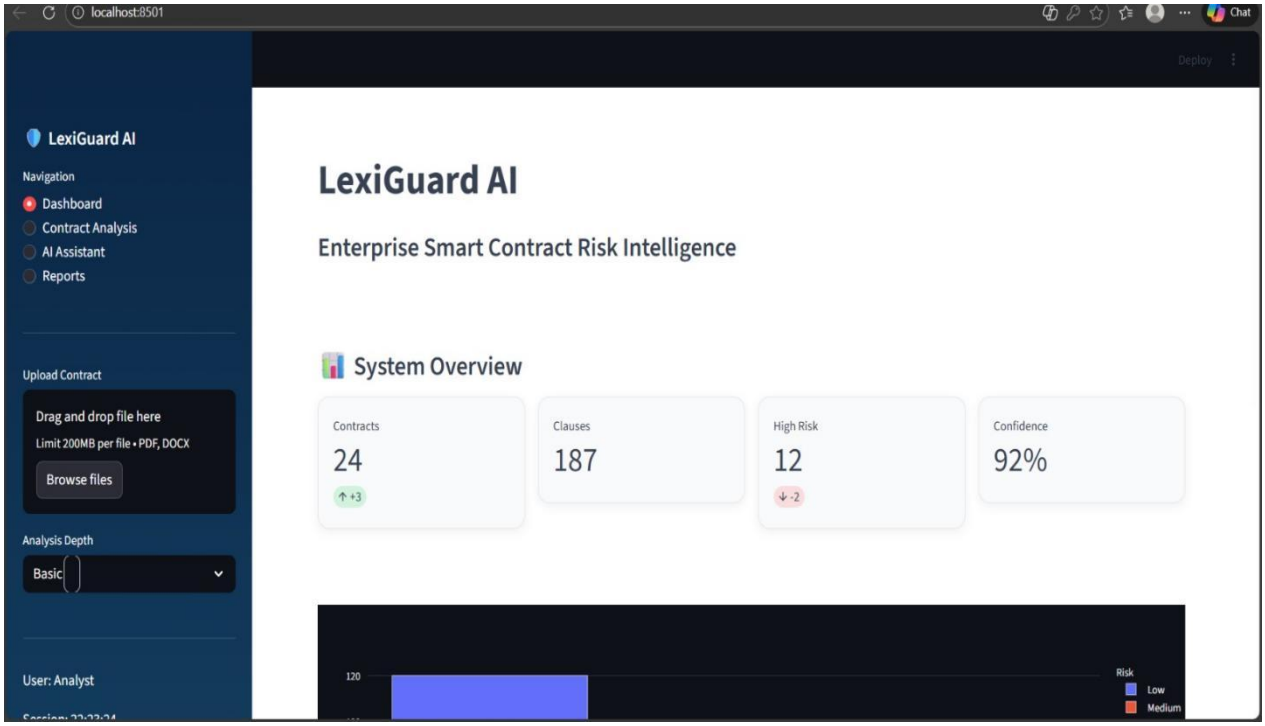
```
        st.dataframe(df,use_container_width=True)

        risk_count = df["Risk Level"].value_counts().reset_index()
        risk_count.columns=["Risk","Count"]

        fig = px.pie(risk_count,names="Risk",values="Count")
        st.plotly_chart(fig,use_container_width=True)

    else:
        st.info("Upload a contract to analyze")

# ---------------- AI ASSISTANT ----------------
elif page == "AI Assistant":

    st.subheader("AI Legal Assistant")

    if client is None:
        st.error("Add OPENAI_API_KEY")
        st.stop()

    if "messages" not in st.session_state:
        st.session_state.messages = []

    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.write(msg["content"])

    prompt = st.chat_input("Ask about contract law...")

    if prompt:

        system_prompt = """
        You are LexiGuard AI, a contract law assistant.
        Answer only contract-related questions.
```

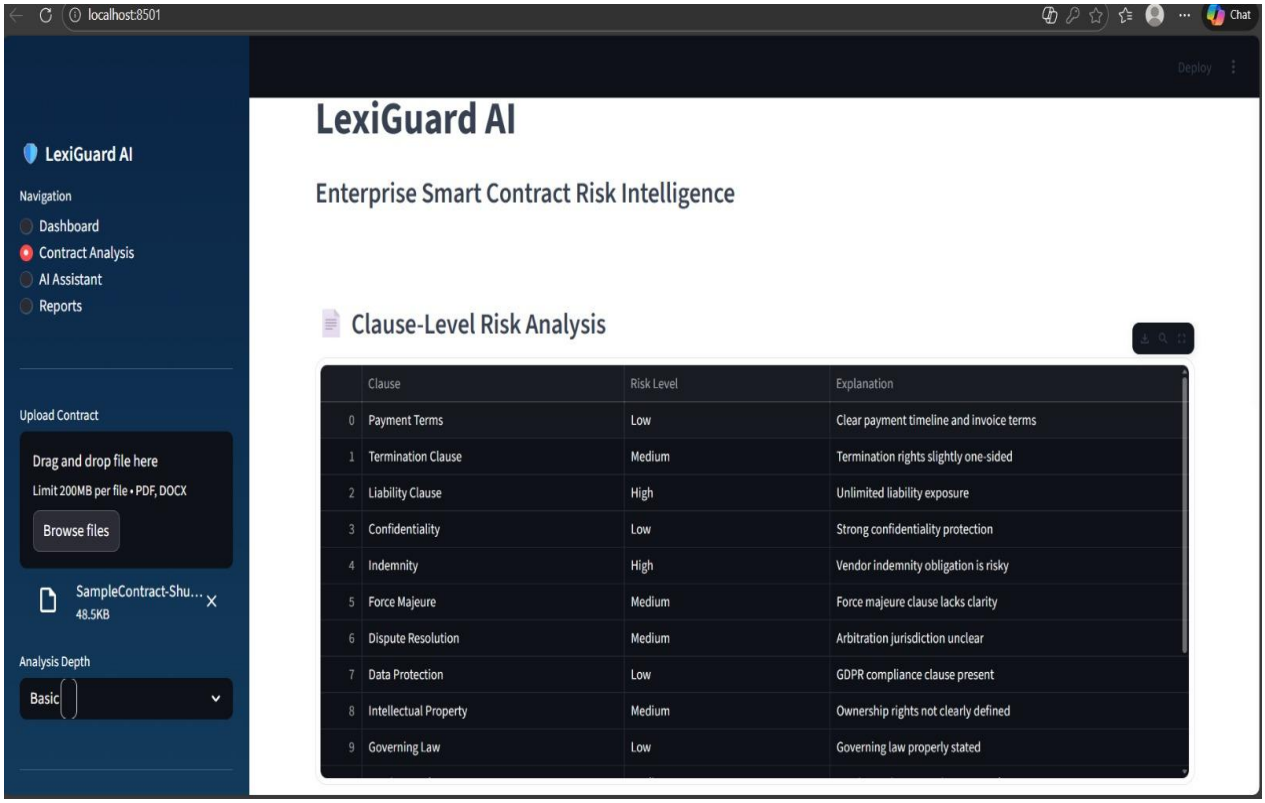
```
        Provide simple explanations and examples.
        """

        st.session_state.messages.append(
            {"role":"user","content":prompt}
        )

        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role":"system","content":system_prompt},
                *st.session_state.messages
            ]
        )

        reply = response.choices[0].message.content

        st.session_state.messages.append(
            {"role":"assistant","content":reply}
        )

        with st.chat_message("assistant"):
            st.markdown(reply)

# ---------------- REPORTS ----------------
elif page == "Reports":

    st.subheader("Export Reports")

    def generate_pdf():
        os.makedirs("reports",exist_ok=True)

        path = "reports/LexiGuard_Report.pdf"
```

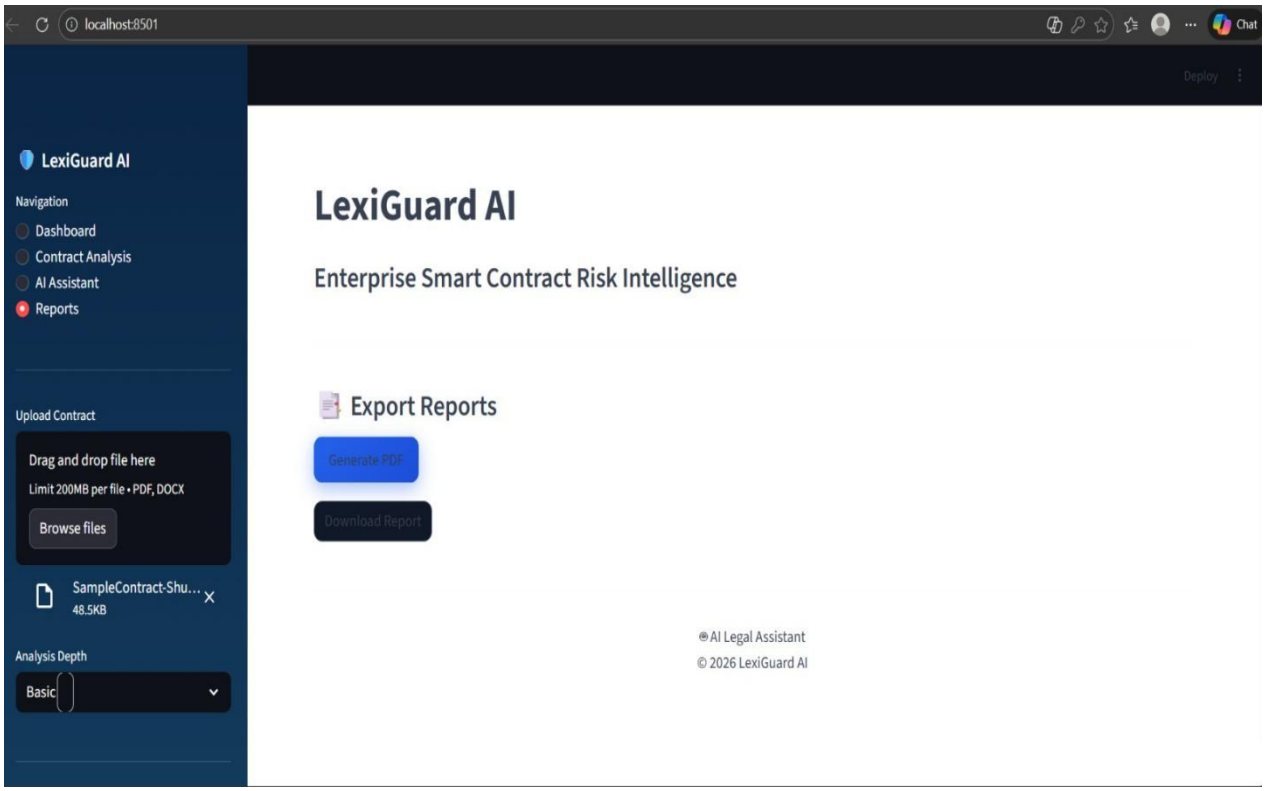
```
        c = canvas.Canvas(path,pagesize=A4)

        c.drawString(50,800,"LexiGuard AI Risk Report")
        c.drawString(50,770,f"Analysis Level: {analysis_level}")
        c.drawString(50,740,"Moderate legal exposure detected")

        c.save()

        return path
    if st.button("Generate PDF"):

        file_path = generate_pdf()

        with open(file_path,"rb") as f:
            st.download_button(
                "Download Report",
                f,
                file_name="LexiGuard_Report.pdf"
            )

st.markdown("---")
st.markdown("© 2026 LexiGuard AI")
```

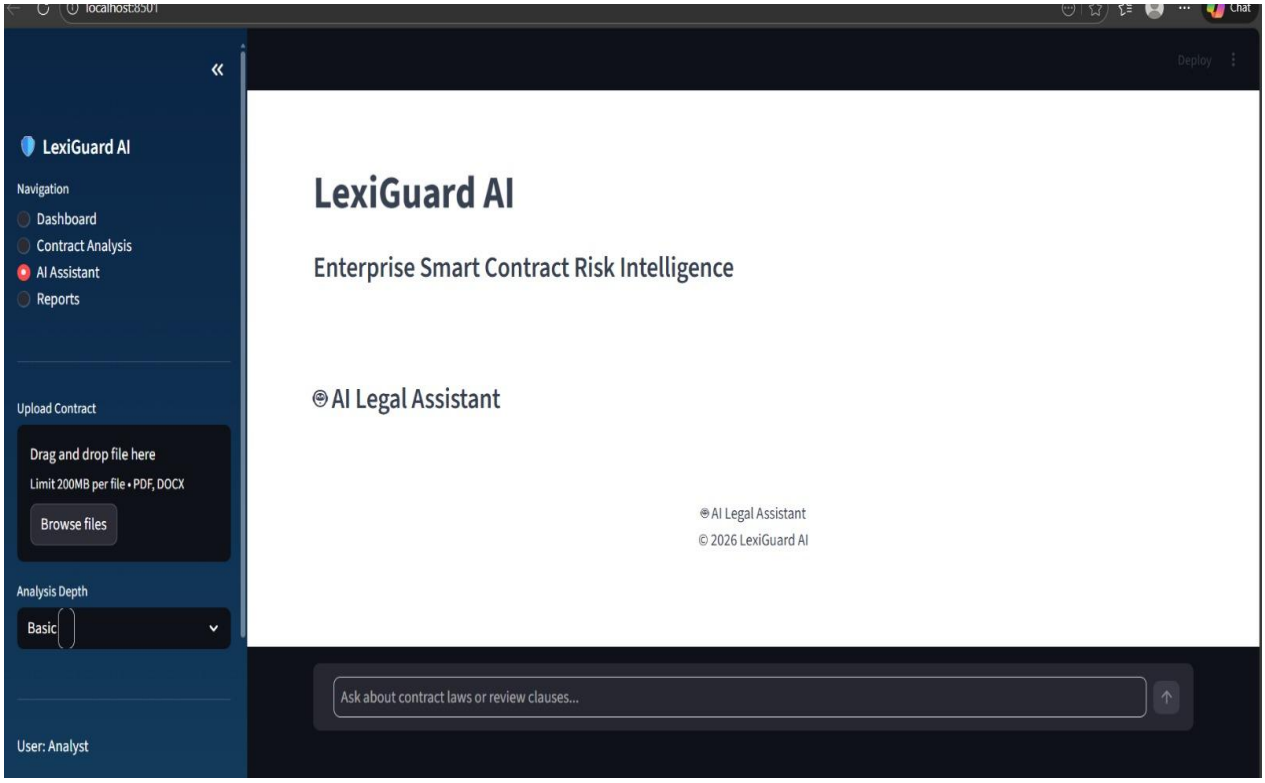
B. SCREENSHOTS

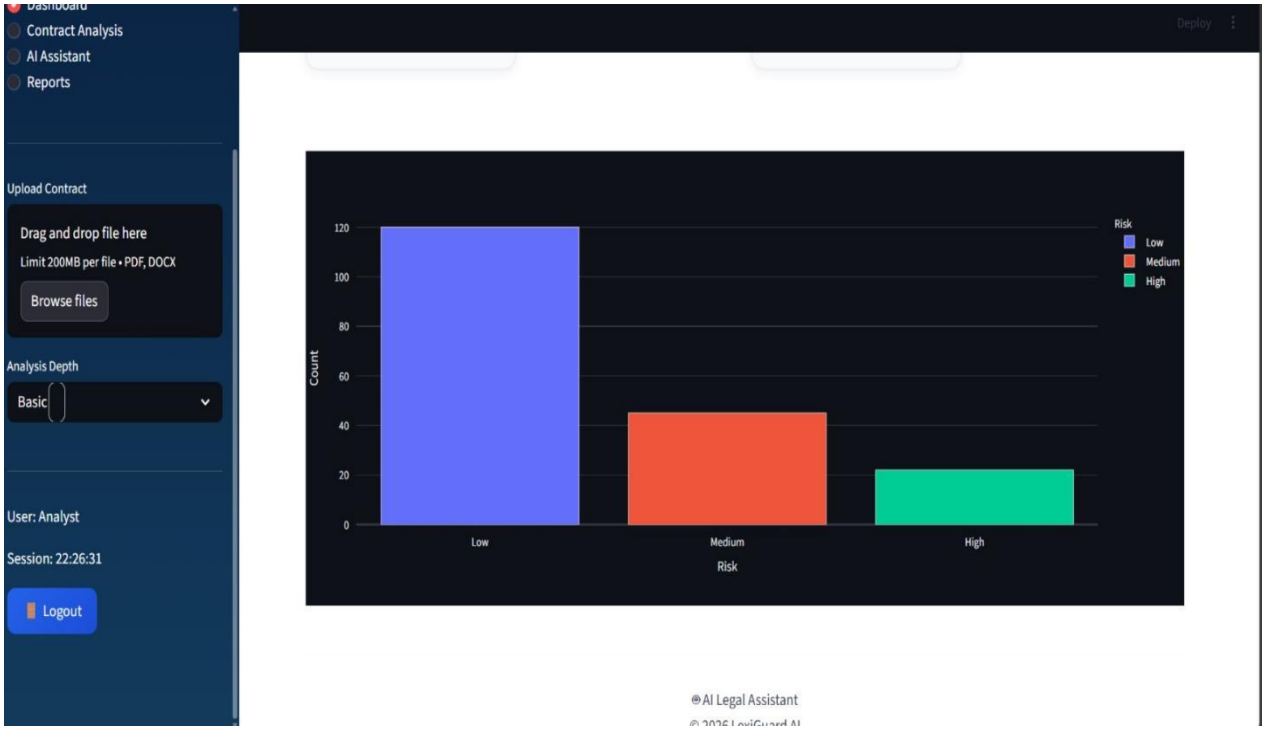





localhost:8501
Chat
Deploy
LexiGuard AI
Navigation
Dashboard
Contract Analysis
AI Assistant
Reports
Upload Contract
Drag and drop file here
Limit 200MB per file • PDF, DOCX
Browse files
SampleContract-Shu...
48.5KB
Analysis Depth
Basic
LexiGuard AI
Enterprise Smart Contract Risk Intelligence
Clause-Level Risk Analysis
Clause | Risk Level | Explanation
0 Payment Terms | Low | Clear payment timeline and invoice terms
1 Termination Clause | Medium | Termination rights slightly one-sided
2 Liability Clause | High | Unlimited liability exposure
3 Confidentiality | Low | Strong confidentiality protection
4 Indemnity | High | Vendor indemnity obligation is risky
5 Force Majeure | Medium | Force majeure clause lacks clarity
6 Dispute Resolution | Medium | Arbitration jurisdiction unclear
7 Data Protection | Low | GDPR compliance clause present
8 Intellectual Property | Medium | Ownership rights not clearly defined
9 Governing Law | Low | Governing law properly stated


localhost:8501
Chat
Deploy
LexiGuard AI
Navigation
Dashboard
Contract Analysis
AI Assistant
Reports
Upload Contract
Drag and drop file here
Limit 200MB per file • PDF, DOCX
Browse files
SampleContract-Shu...
48.5KB
Analysis Depth
Basic
LexiGuard AI
Enterprise Smart Contract Risk Intelligence
Export Reports
Generate PDF
Download Report
AI Legal Assistant
© 2026 LexiGuard AI


LexiGuard AI
Navigation
Dashboard
Contract Analysis
AI Assistant
Reports
Upload Contract
Drag and drop file here
Limit 200MB per file • PDF, DOCX
Browse files
Analysis Depth
Basic
User: Analyst
LexiGuard AI
Enterprise Smart Contract Risk Intelligence
AI Legal Assistant
AI Legal Assistant
© 2026 LexiGuard AI
Ask about contract laws or review clauses...


Contract Analysis
AI Assistant
Reports
Upload Contract
Drag and drop file here
Limit 200MB per file • PDF, DOCX
Browse files
Analysis Depth
Basic
User: Analyst
Session: 22:26:31
Logout
Count
120
100
80
60
40
20
0
Low
Medium
High
Risk
Risk
Low
Medium
High
AI Legal Assistant